

PinIT Career OS — Full-Stack Web Certification Track

Software Requirements Specification (SRS) and Step-by-Step Build Instructions

Field	Value
Document	PINIT_WEB_FULLSTACK_SRS
Version	1.0
Date	1 October 2026
Written for	The Antigravity coding agent (and the owner, for review)
Written by	Claude Code, from the project conversation, two audits of Antigravity's internship work, and a scan of <code>main</code> at commit <code>ab5d2b82</code>
Repository	<code>Vinay-N-Kashyap/Present-Career-os</code>
Working branch	<code>main-dis3ku</code> (merge to <code>main</code> only through a pull request)
Scope	Phase 0: finish the internship fixes. Then the web_fullstack track of the 1, 3, 6, 9 and 12-month plans: courses, lessons, checked tasks, capstones and internships

Contents

PART A — READ THIS FIRST

- A1. How to use this document (for the Antigravity agent)
- A2. The standard checks (after every task)
- A3. Mistakes the agent must avoid (all of these happened before in this project)
- A4. How merging works
- A5. Report format (after every task)

PART B — BACKGROUND

- B1. What exists today
- B2. Audit of Antigravity's internship work (why Phase 0 exists)
- B3. The web track today (scan of `main`)
- B4. Owner decisions for this document (answered)

PART C — REQUIREMENTS

- C1. Target web 12-month plan
- C2. Course standard (applies to every web course)
- C3. Content of the three new courses and Node
- C4. Real checking infrastructure
- C5. Web internship (same 5 tiers)

C6. Web track tests (new)

C7. Web capstones

C8. Acceptance criteria

PART D — STEP-BY-STEP TASKS

Phase 0 — Finish the internship fixes (do these first)

Phase 1 — Real checking infrastructure

Phase 2 — Web track test harness

Phase 3 — Courses (in plan order)

Phase 4 — Plan wiring

Phase 5 — Web capstones

Phase 6 — Web internship

Phase 7 — Launch checks

PART E — REFERENCE

E1. Standard commands

E2. Key files

E3. Owner's short summary (non-technical)

PART A — READ THIS FIRST

A1. How to use this document (for the Antigravity agent)

This document is both a requirements specification and a work plan. Part D splits the job into **small numbered tasks** (W-00-1, W-01, W-02, ...). Follow these rules exactly:

1. **Do one task at a time, in order.** Do not start the next task until the current one passes all its checks.
2. **Read every file listed under "Read first" before you change anything.**
3. **Change only the files the task names.** If another file seems to need changing, stop and ask the owner.
4. **Run the standard checks (A2) after every task.** If one fails, fix your change. Never skip, delete, silence or weaken a test (this includes adding an early `return` inside a test).
5. **Commit after every task** with the message given, on branch `main-dis3ku`, then `git push -u origin main-dis3ku`.
6. **Never push to `main`.** The owner (or Claude Code) merges through a pull request after the "Web app tests" check passes.
7. **Never invent facts, data, companies, reviews or results.** Report only what you ran and saw.
8. **Stop at the end of every phase** and report to the owner in the format in A5. Wait for "continue".
9. If a task is marked **BLOCKED**, skip it and continue with the next unblocked task.
10. If something in this document does not match the real code, **stop and ask**. Do not guess.

A2. The standard checks (after every task)

```
npx tsc --noEmit      # must exit 0 with no output
npm test              # every test must pass; the total must never go down (383 at the
time of writing)
npx next build        # must finish without errors
git status --short    # must NOT list tsconfig.tsbuildinfo in your commit (restore it
with: git checkout -- tsconfig.tsbuildinfo)
```

A3. Mistakes the agent must avoid (all of these happened before in this project)

#	Mistake	What to do instead
1	Declaring the same <code>const</code> twice in one scope	Use another name or reuse the variable
2	Duplicate keys in an inline style object	One key per property
3	Using <code>any</code>	Use or import the real type

#	Mistake	What to do instead
4	Trusting data from the browser (passed, score, progress)	Only the server decides and records results
5	Sending hidden tests or reference solutions to the browser	Select explicit columns; use the <code>toClient</code> mappers
6	Hand-written tasks or projects used in production	Tasks are generated by AI at run time (owner rule). Fixed data belongs only in <code>tests/</code>
7	Using AI output without running it	Every generated task passes the V1-V7 validation (internship SRS §C12)
8	Tests that replace the real sandbox with a fake, so real bugs are never seen	Every grader needs at least one test that runs the real sandbox with a correct answer, a wrong answer and a cheat attempt
9	"Passed" decided by looking for a text marker the student can print	Decide pass/fail by exit code and a secret per-run marker the student never sees
10	Checks that only count rows or check types (any wrong answer with the right shape passes)	Compare real values
11	Tasks whose answer is "return a constant" (e.g. <code>return 'toast'</code>)	Every task needs input → processing → output, checked on several inputs
12	Tests written for a tool that is not installed (Vitest)	Use <code>node:test</code> + <code>node:assert/strict</code> , like every other test in <code>tests/</code>
13	Claiming results that were not reproduced (fake test counts, "end-to-end verified", files that do not exist)	Paste the real command output in your report
14	Building BLOCKED tasks	Skip them
15	Pushing directly to <code>main</code>	Push only to <code>main-dis3ku</code>
16	Committing <code>tsconfig.tsbuildinfo</code>	Restore it before committing
17	Editing the 24-month plan (<code>plan-24m-master</code>)	Never touch it (owner rule)
18	Editing old migration files	Add a new migration file
19	Quest id prefixes containing <code>java</code> , <code>sql</code> or <code>python</code> for web courses	<code>adaptCodeForSandbox</code> in <code>src/app/quests/lesson/hooks/useLessonEngine.ts</code> rewrites such code. Use prefixes like <code>node-web</code>
20	Wrong expected values in tasks or lessons (more than 20 were found in earlier AI-written courses)	Produce every expected value by running the reference answer; never type it by hand

A4. How merging works

- Open a PR from `main-dis3ku` to `main`.
- **"Web app tests (database rules and API behaviour)"** must pass.
- **"Build, Test & Lint"** always fails, because it runs `pytest voice-service/tests` and that folder does not exist. This is old and unrelated; leave it alone and mention it once in the PR.

- Merge method: **merge** (not squash or rebase). Only the owner or Claude Code merges.

A5. Report format (after every task)

```
TASK: W-xx <title>
FILES CHANGED: <list>
CHECKS: tsc ok | npm test <passed>/<total> | build ok      (paste the real numbers)
COMMIT: <short hash> (pushed to main-dis3ku)
NOTES: <anything unexpected, or "none">
```

PART B — BACKGROUND

B1. What exists today

- **Python track (python_ai):** complete. 12 distinct courses for the 12-month plan, each with 30 days, 30 long lessons of 20–30 minutes, and 60 practice tasks that are really checked. This is the **pattern to copy**.
- **Internship program:** built by Antigravity (tasks T-01...T-40 of docs/internship-srs/PINIT_INTERNSHIP_SRS.md), **Python only**, all switches off. Two audits found serious problems; see B2.
- **Web track (web_fullstack):** far behind the Python track; see B3.

B2. Audit of Antigravity's internship work (why Phase 0 exists)

First audit (commit c9a3d3fe) found: no student could pass a ticket (the guard blocked `import sys`); all 8 fixed Tier 2 tasks broken; 11 test files used Vitest, which is not installed; hand-written fallbacks; BLOCKED tasks built; pushed straight to `main`; claims that could not be reproduced.

Second audit (commit ab5d2b82, "audit remediation complete"). Checked by running the real code:

Check	Result
<code>npx tsc --noEmit</code>	passes
<code>npm test</code>	383 / 383 pass (Antigravity reported 371)
Correct Python answer submitted	passes (the first bug is fixed)
Wrong Python answer submitted	fails with "Hidden check 2 failed" (correct)
Wrong answer + <code>print("__PINIT_TESTS_PASSED__")</code>	passes: cheating hole
Wrong SQL query <code>SELECT * FROM <table> LIMIT n</code>	passes on all 4 Tier 2 SQL tasks: checks only count rows
Tier 2 SQL starters	2 of 4 already pass (validator step V5 rejects them)

Still open:

1. **Cheating hole.** The hidden tests catch their own failures and print a message, and "passed" is decided by looking for the text `__PINIT_TESTS_PASSED__`. Student code runs first, so a student who prints that text passes every ticket. Files:
`src/lib/internships/submission.ts` (`instrumentHiddenPythonTests`, `executeTicketCode`),
`src/lib/server/pythonSandbox.ts` (`PASS_SENTINEL` is the same text).
2. **SQL checks only count rows.** `src/lib/internships/tier2Tasks.ts`, SQL tasks seq 2, 4, 6, 8.

3. **Hand-written fallbacks still used in production:** `getDeterministicProductBrief` (`src/lib/internships/productBrief.ts`) and `getDeterministicTier2Tasks` (`src/lib/internships/tier2Tasks.ts`).
4. **An unrelated test was weakened.** `tests/python_track_lessons.test.ts` now silently returns when `python3` is missing, so it passes while checking nothing, and it adds `any`.
5. **The start route never checks for an existing active internship.** `src/app/api/internship/start/route.ts` passes `activeInternship: false`.
6. **No test runs a correct Python answer through `executeTicketCode` with the real sandbox.**
7. **Process:** pushed straight to `main` again, and committed `tsconfig.tsbuildinfo`.

Fixed correctly: `sys` removed from the instrumentation; 11 test files converted to `node:test`; `vitest.config.mjs` removed; all internship switches still `false`.

B3. The web track today (scan of `main`)

Web 12-month plan (`src/lib/data/crashPlansData.ts`, `plan-12m-fellow`, `modulesByTrack.web_fullstack`):

M	Course id	Problem
1	<code>course-react-web</code>	OK: 30 long lessons; 1 constant task (Day 1 <code>pageTitle()</code>)
2	<code>course-fullstack-js</code>	A 120-day course (240 tasks) in a one-month slot; days 28–47 repeat React and days 58–67 repeat SQL; no long lessons
3	<code>course-database-eng</code>	OK (30 long lessons, shared with Python)
4	<code>course-dsa-optim</code>	No long lessons
5	<code>course-devops-cicd</code>	No long lessons
6	<code>course-cloud-native</code>	No long lessons
7	<code>course-distributed-sys</code>	No long lessons; Day 30 stub task
8	<code>course-cybersecurity</code>	No long lessons; 30 of 60 tasks return a constant
9	<code>course-ai-eng</code>	No long lessons; Day 30 stub task
10	<code>course-cloud-native</code>	Repeat of Month 6 (adds zero lessons; <code>getCrashCourseCurriculum</code> in <code>src/lib/courses/crashCourseProgress.ts</code> skips repeated quest ids)
11	<code>course-distributed-sys</code>	Repeat of Month 7
12	<code>course-ai-prompt-literacy</code>	Title says "Production AI Deployment", but the course is "Everyday AI Literacy & Prompt Engineering"; no long lessons; 29 constant tasks

The 6-month plan also uses `course-design-systems` (Month 6): no long lessons and **30 of 60 tasks return a constant**. Example: `src/lib/data/design30DayData.ts` asks for

`getHighestZIndexLayerName()` returning `'toast'`.

How web code is checked today:

- JavaScript runs only in the browser, in a sandboxed iframe plus Web Worker (`src/lib/code/sandbox/sandboxedIframeRunner.ts`), started from `src/lib/code/codeRunner.ts` and `src/lib/code/runners/jsTaskScript.ts`.
- Tests replay the tasks in Node's `vm` (`tests/js_practice_tasks.test.ts`), with answers in the shared file `tests/fixtures/practice_solutions.json`.
- **There is no TypeScript compiler, no JSX/React rendering, no HTML/CSS checking, and no forbidden-API guard for JavaScript.** `esbuild` is already a dependency but no runner uses it. `react`, `react-dom`, `parse5` and `postcss` are in `node_modules`, but only `react/react-dom` are direct dependencies.
- `CodeLanguage` in `src/lib/code/types.ts` is `'javascript' | 'python' | 'sql' | 'java' | 'cpp'`.
- `resolveQuestLanguage` (`src/components/quests/workspace/useWorkspaceState.ts`) maps TypeScript to JavaScript.
- The only server-side JavaScript sandbox is `src/lib/server/codeWarsSandbox.ts` (`node:vm` in a worker thread with a ban list, `validateVmCodeSecurity`). It serves Code Wars only.
- `tests/react_long_lessons.test.ts` runs lesson examples as plain JavaScript in `vm`; real JSX is shown read-only in `projectCode`.

Internship is Python-only in about 10 places:

- `src/lib/internships/eligibility.ts` (track check)
- `src/app/quests/components/internship/InternshipPanel.tsx` ("exclusive to the Python track")
- `src/lib/internships/tiers.ts` (keyed by tier only, Python names and courses)
- `types.ts`, `generateTask.ts`, `validateTask.ts`, `submission.ts` (`language: 'python' | 'sql'`)
- `toClient.ts` (forces `'python'` and a default track of `'python_ai'`)
- migration `supabase/migrations/20261001_internship_program.sql` (track default `'python_ai'`; language CHECK IN (`'python'`, `'sql'`))

B4. Owner decisions for this document (answered)

1. **Keep the current web month order and fix it** (do not rebuild from the master curriculum doc).
2. **Real checks** for TypeScript, JSX/React, HTML and CSS.
3. **Web internship with the same 5 tiers** as Python.
4. **Internship fixes first**, as Phase 0 of this document.

Standing rules from earlier: every lesson is a 20–30 minute class; tasks are real and checked; projects and internship tasks are AI-generated (no hard-coding); do not build the 24-month course; reports are short and honest.

PART C — REQUIREMENTS

C1. Target web 12-month plan

M	Course id (quest prefix)	Status	Required work
1	course-react-web (react-basics)	keep	Replace the Day-1 constant task; add real JSX/TSX tasks once the new checker exists (≥ 10 of 60 tasks render components)
2	course-node-web (node-web), new	new	30-day Node.js + Express + TypeScript backend course. Plans 3m, 6m, 9m and 12m point Month 2 here. course-fullstack-js stays unchanged for the 24-month plan
3	course-database-eng (sql-mastery)	done	none
4	course-dsa-optim (dsa-optim)	upgrade	+ 30 long lessons; check for constant tasks
5	course-devops-cicd (devops)	upgrade	+ 30 long lessons
6	course-cloud-native (cloud)	upgrade	+ 30 long lessons
7	course-distributed-sys (dist)	upgrade	+ 30 long lessons; fix the Day-30 stub
8	course-cybersecurity (cyber)	upgrade	+ 30 long lessons; rewrite the 30 constant tasks
9	course-ai-eng (ai)	upgrade	+ 30 long lessons; fix the Day-30 stub
10	course-sre-web (sre-web), new	new	Multi-Cloud Reliability & SRE in TypeScript
11	course-stream-web (stream-web), new	new	High-Throughput Streaming in TypeScript
12	course-aideploy-web (aideploy-web), new	new	Production AI Deployment in TypeScript
6m M6	course-design-systems (design)	upgrade	+ 30 long lessons; rewrite the 30 constant tasks with real HTML/CSS/React checks

Plans after the change (web_fullstack):

- **1m:** M1.
- **3m:** M1–M3.
- **6m:** today's order with only Month 2 changed: react-web, node-web, database-eng, devops-cicd, cloud-native, design-systems.

- **9m:** M1-M9.
- **12m:** M1-M12.

Keep each plan's existing month titles, except Month 2 (use the new course's title) and Months 10-12 (use the titles in C3).

C2. Course standard (applies to every web course)

Each course must meet the same bar as the Python courses:

Item	Requirement
Days	30, in <code>src/lib/data/<name>WebDays.ts</code> as <code>DayConfig[]</code> (day, title, desc, syllabus). For upgraded courses, reuse the existing day titles
Practice tasks	2 per day (60), in <code>src/lib/data/<name>Web30DayData.ts</code> built with <code>buildEnrichedDayQuests(prefix, day, cfg)</code> from <code>src/lib/data/curriculumEnricher.ts</code>
Answers	<code>tests/fixtures/<name>_web_solutions.json</code> (quest id → answer code). Never shipped to students
Task quality	The reference answer passes; the starter fails; lazy answers fail (<code>true</code> , <code>false</code> , <code>0</code> , <code>[]</code> , <code>''</code> , <code>null</code> , <code>{}</code>); constant answers fail (each check uses ≥ 3 different inputs, and at least one expected value is computed from the input); no forbidden APIs (C4)
Long lessons	30, in <code>src/lib/data/<name>WebLongLessons.ts</code> , registered in <code>src/lib/data/longLessons.ts</code> (<code>LONG_LESSON_SOURCES</code> + <code>LONG_LESSON_LANGUAGE</code>)
Lesson shape	Exactly 6 parts; each part has 8-12 <code>say</code> lines, a real-world example, runnable code, the real output, <code>codeNotes</code> , a <code>tryIt</code> question and a 3-option check. Plus <code>goal</code> , <code>recap</code> , <code>summary</code> (5 lines) and <code>projectStep</code>
Lesson length	≥ 9.2 spoken minutes = $\geq 1,105$ words across <code>say</code> + <code>example</code> + <code>tryIt</code> + <code>check.why</code> (about 120 spoken words per minute). With examples and quizzes, each class is 20-30 minutes
Lesson truth	Every <code>output</code> is produced by running the code; every <code>tryIt</code> is consistent with that output; every number in a <code>say</code> line that describes an example matches the output
Registration	<code>coursesData.ts</code> (entry with id, title, desc, difficulty, durationWeeks 6, icon, quests); <code>RUNNER_LABELS</code> in <code>src/app/quests/lesson/hooks/useLessonEngine.ts</code> ; <code>COURSE_TO_ROLE</code> in <code>src/app/quests/components/useQuestProgression.ts</code> ; <code>src/lib/data/courseNotesRegistry.ts</code> ; the web track tests (C6)

C3. Content of the three new courses and Node

Topics are listed by day group; each day gets 2 tasks that exercise that day's idea.

course-node-web — Node.js & TypeScript Backend Engineering (Month 2):

- Days 1-5: the Node runtime and event loop; modules; TypeScript basics for the backend (types, interfaces, generics, narrowing); `async/await` and errors.
- Days 6-10: the HTTP protocol (methods, status codes, headers); building request handlers as pure functions; routing tables; query strings and URL parsing; request body validation with Zod-style schemas.

- Days 11–15: middleware chains; error handling with RFC 7807 problem details; structured logging; configuration and environment variables; pagination and filtering.
- Days 16–20: authentication (password hashing concepts, sessions vs tokens, JWT structure and expiry checks); authorization rules; rate limiting; CORS; input sanitising.
- Days 21–25: data access layers; repository pattern over an in-memory store; transactions (concept); caching with TTL; idempotency keys.
- Days 26–30: testing handlers; OpenAPI descriptions; background jobs and retries; graceful shutdown and health checks; capstone: a small REST API.

course-sre-web — Multi-Cloud Reliability & SRE in TypeScript (Month 10):

- Days 1–5: SLIs, SLOs, error budgets and burn rates; availability math (serial and parallel).
- Days 6–10: infrastructure as code as data (plan/diff/apply of resource maps, drift detection); multi-region and failover planning.
- Days 11–15: load balancing algorithms; health checks; retries with backoff and jitter; circuit breakers; bulkheads and timeouts.
- Days 16–20: observability (metrics, percentiles, histograms, log parsing, trace spans); alert rules and noise reduction.
- Days 21–25: incident response (severity, timelines, MTTD/MTTR); postmortems; capacity planning; cost models.
- Days 26–30: chaos experiments; deployment strategies (blue/green, canary analysis); runbooks as code; capstone: a reliability scorecard.

course-stream-web — High-Throughput Streaming in TypeScript (Month 11):

- Days 1–5: event logs and offsets; producers/consumers; partitioning by key; ordering guarantees.
- Days 6–10: consumer groups and rebalancing; commit strategies; at-most/at-least/exactly-once; deduplication.
- Days 11–15: backpressure and bounded queues; batching and linger; compression trade-offs; throughput vs latency math.
- Days 16–20: stream processing (map/filter/reduce over streams); tumbling, sliding and session windows; watermarks and late events.
- Days 21–25: joins (stream-table, stream-stream); state stores; checkpoints and recovery; schemas and evolution.
- Days 26–30: dead-letter queues; replay; monitoring lag; capacity planning; capstone: a windowed analytics pipeline.

course-aideploy-web — Production AI Deployment in TypeScript (Month 12):

- Days 1–5: calling model APIs safely (timeouts, retries, typed responses); token counting and cost estimates; prompt templates.
- Days 6–10: streaming responses (parsing server-sent events); request batching; semantic caching with similarity thresholds; rate limiting per user.
- Days 11–15: model routing and fallbacks; latency budgets and percentiles; quantization and model-size trade-offs (as calculations); GPU/CPU capacity math.
- Days 16–20: retrieval-augmented generation plumbing (chunking, top-k merge, citations); output schema validation; guardrails (input validation, prompt-injection patterns, PII masking).

- Days 21–25: evaluation harnesses and regression gates; A/B comparisons; feedback logging; drift detection.
- Days 26–30: observability for AI calls; cost dashboards; incident handling; deployment checklists; capstone: a guarded, cached, observable AI endpoint.

C4. Real checking infrastructure

Id	Requirement
CHK-1	CodeLanguage gains four values: <code>typescript</code> , <code>tsx</code> , <code>html</code> and <code>css</code> . <code>resolveQuestLanguage</code> returns them for quests that declare them (a <code>language</code> field on the quest/day config). Existing JavaScript quests are unchanged
CHK-2	TypeScript/TSX compile: in the browser with <code>esbuild-wasm</code> (add it as a dependency; load the <code>wasm</code> file from <code>public/</code> , copied by a script like <code>scripts/utils/copy-pyodide.cjs</code>); in Node tests with the existing <code>esbuild</code> package. Compile errors are shown to the student in plain words, with the line number
CHK-3	React checks without a DOM: a prebuilt runtime bundle (<code>React + react-dom/server</code>), built by a script into <code>public/sandbox/react-runtime.js</code> , is placed in front of the student code in the Worker. Checks call <code>renderToStaticMarkup(<Component {...props} />)</code> and compare HTML output for several prop sets
CHK-4	HTML/CSS checks: a small bundled parser (<code>parse5</code> for HTML, <code>postcss</code> for CSS; add both as direct dependencies). Helper functions for checks: <code>queryAll(html, selector)</code> , <code>attr(el, name)</code> , <code>text(el)</code> , <code>cssRules(css)</code> , <code>cssValue(css, selector, property)</code> , plus accessibility helpers (every <code>img</code> has <code>alt</code> , every input has a label, headings in order)
CHK-5	JS/TS guard <code>src/lib/code/js/jsGuard.ts</code> (like <code>src/lib/code/python/pythonGuard.ts</code>): refuse <code>fetch</code> , <code>XMLHttpRequest</code> , <code>WebSocket</code> , <code>importScripts</code> , <code>eval</code> , <code>new Function</code> , <code>Function</code> , <code>dynamic import</code> , <code>require</code> , <code>process.</code> , <code>globalThis</code> , <code>constructor.constructor</code> , <code>__proto__</code> , <code>document.cookie</code> , <code>localStorage</code> , <code>indexedDB</code> . It applies to student code and tests in the browser runner, in the server grader and in the tests
CHK-6	Server grader <code>src/lib/server/jsSandbox.ts</code> for internship hidden tests: <code>node:vm</code> inside <code>worker_threads</code> with a timeout (reuse the approach of <code>src/lib/server/codeWarsSandbox.ts</code>), <code>esbuild</code> transform for TS/TSX, <code>jsGuard</code> first. Pass/fail is decided by the worker's exit status and a random per-run marker created on the server and never shown to the student. Output is trimmed and never contains hidden test source
CHK-7	<code>adaptCodeForSandbox</code> in <code>useLessonEngine.ts</code> must not rewrite code for quests whose language is set (<code>typescript/tsx/html/css/javascript</code>); add a test
CHK-8	The lesson page runs <code>typescript / tsx</code> lesson examples through the same compile step; <code>LongLessonLanguage</code> gains <code>'typescript'</code>

C5. Web internship (same 5 tiers)

Tier	Plan	Web name (default, owner may rename)	Work
t1_job_sim	1m	Web Developer Job Simulation	5 AI-generated tickets: React component (TSX), component bug fix, form validation function, refactor, small feature. Checked with hidden tests on the server grader (CHK-6)

Tier	Plan	Web name (default, owner may rename)	Work
t2_virtual_team	3m	Virtual Internship - Full-Stack	Team backlog for a React + Node + PostgreSQL app; weekly checked tasks (1 TypeScript/TSX + 1 SQL); PR links; stand-ups; demo
t3_project	6m	Project Internship - Web Products	BLOCKED (owner decisions D1 mentors, D2 partners)
t4_industry	9m	Verified Industry Internship	BLOCKED (D1, D2)
t5_fellowship	12m	Verified Fellowship + Placement	BLOCKED (D1, D2)

Rules:

- Same eligibility, validation (V1-V7), certificates and honesty rules as the Python internship (docs/internship-srs/PINIT_INTERNSHIP_SRS.md , Parts C3-C13).
- tiers.ts becomes per-track: INTERNSHIP_TIERS[track][tier] .
- New task languages typescript and tsx , allowed by a **new** migration.
- The per-tier switches stay false until the owner approves each one.

C6. Web track tests (new)

Test file	What it proves
tests/web_track_tasks.test.ts	For every web course (C1): 60 tasks; each reference answer passes; each starter fails; lazy answers fail; constant answers fail ; jsGuard finds nothing in answers or tests; TS/TSX tasks compile
tests/web_track_long_lessons.test.ts	For every web course: 30 lessons, days 1-30, 6 parts each, ≥ 9.2 spoken minutes, every example runs and prints exactly its output , titles match the DayConfig titles
tests/web_track_plans.test.ts	Web 12m has 12 distinct courses; $1m < 3m < 9m < 12m$ by month; every web course used by 1m-12m has 30 long lessons and 60 tasks; plan-24m-master is unchanged (compare its module list with a snapshot)
tests/js_guard.test.ts , tests/ts_compile.test.ts , tests/react_render_check.test.ts , tests/html_css_check.test.ts , tests/js_sandbox.test.ts	Infrastructure; each includes a correct answer, a wrong answer and a cheat attempt

C7. Web capstones

Plans 3m-12m get web capstone sprint wording in capstoneSprintsByTrack.web_fullstack (src/lib/data/crashPlansData.ts , read by getCapstoneSprints in src/lib/courses/capstoneSprints.ts). Each plan's flagship build (flagshipBuildByTrack.web_fullstack) may only list technology its courses teach; add a test like the 1-month check in tests/crash_plan_promises.test.ts .

C8. Acceptance criteria

1. Phase 0: the cheat attempt fails; wrong SQL queries fail; no hand-written fallbacks in production; `tests/python_track_lessons.test.ts` restored; the active-internship check works; real-sandbox tests exist; the work arrived through a PR.
 2. Web 12m has 12 distinct courses, each with 30 long lessons (≥ 9.2 spoken minutes) and 60 real, checked tasks.
 3. Students write and run real TypeScript, React (TSX), HTML and CSS, checked with real renders and parsers.
 4. The web internship Tiers 1-2 work end to end with the server grader; Tiers 3-5 stay BLOCKED.
 5. `npx tsc --noEmit`, `npm test` and `npx next build` pass; `plan-24m-master` is unchanged; all internship switches remain `false`.
-

PART D — STEP-BY-STEP TASKS

Every task: **Read first** → **Change** → **Steps** → **Must not** → **Verify (standard checks + extra)** → **Commit message**. Stop after each phase.

Phase 0 — Finish the internship fixes (do these first)

W-00-1 — Close the cheating hole

- **Read first:** `src/lib/internships/submission.ts`, `src/lib/server/pythonSandbox.ts`, `tests/internship_submission.test.ts`.
- **Change:** `submission.ts`, `pythonSandbox.ts` (only the marker handling), `tests/internship_submission.test.ts`.
- **Steps:**
 1. In `instrumentHiddenPythonTests`, on failure **re-raise**: `raise AssertionError(f"Hidden check {n} failed")` from `None` (do not print and continue).
 2. Give `runPythonInSandbox` an optional `sentinel` argument; `executeTicketCode` passes a fresh random value (`crypto.randomBytes(16).toString('hex')`) for every run; the runner prints that value only after all tests pass.
 3. Pass = exit code 0 **and** the per-run sentinel present. Remove the `__PINIT_TESTS_PASSED__` output check from `executeTicketCode`.
 4. Output shown to students contains only "Hidden check N failed" for hidden failures.
- **Must not:** change `/api/code/run-python` behaviour for courses.
- **Verify:** new tests using the **real** sandbox:
 - (a) a correct `add` passes;
 - (b) a wrong `add` fails with "Hidden check N failed";
 - (c) the wrong `add` plus `print("__PINIT_TESTS_PASSED__")` **fails**;
 - (d) the output never contains any hidden test line.

Skip visibly with `{ skip: 'python3 not installed' }` if `python3` is missing.

- **Commit:** Close the hidden-test sentinel cheat in internship grading

W-00-2 — Make Tier 2 SQL checks compare values

- **Read first:** `src/lib/internships/tier2Tasks.ts`, `src/lib/code/sql/sqlCore.ts` (`runSqlPractice`, `check format`), `src/lib/internships/validateTask.ts`.
- **Change:** `tier2Tasks.ts` (SQL tasks seq 2, 4, 6, 8) and a new test `tests/internship_tier2_sql_checks.test.ts`.
- **Steps:**
 - Rewrite every visible and hidden SQL check to compare real values, for example:


```
SELECT (SELECT array_agg(name ORDER BY name) FROM answer) = ARRAY['Alice Johnson', 'Bob Smith'] AS ok, 'Returns the two active adult patients' AS msg;
```


- Starters must return wrong data, for example `SELECT id, name FROM patients WHERE false;`.
- **Verify:** for each task:
 - `validateGeneratedTask(task, 'sql')` returns ok;
 - the reference passes;
 - `SELECT * FROM <table> LIMIT n` **fails**;
 - the starter fails.
- **Commit:** Make internship SQL checks compare values

W-00-3 — Remove hand-written fallbacks from production

- **Read first:** `src/lib/internships/productBrief.ts`, `src/lib/internships/tier2Tasks.ts`, `src/app/api/internship/team/route.ts`, any caller of `generateTier2MemberTasks`.
- **Steps:**
 1. When AI generation fails after the allowed attempts, return `{ ok: false, reasons }`.
 2. The route sets `status: 'generation_failed'` and shows "We could not prepare your internship right now. Please try again in a few minutes."
 3. Move `getDeterministicProductBrief` and `getDeterministicTier2Tasks` data into `tests/fixtures/` (tests may use them through a fake AI).
- **Verify:** `git grep -n "deterministic-fallback\|getDeterministic" -- src` returns nothing.
- **Commit:** Remove hand-written internship fallbacks from production

W-00-4 — Restore the weakened Python lesson test

- **Change:** `tests/python_track_lessons.test.ts` only.
- **Steps:** Restore the original check. If you need to support machines without `python3`, check once at the top, and mark the tests `{ skip: 'python3 not installed' }` so the skip is visible. No `any`, no early `return` inside a test.
- **Commit:** Restore strict Python lesson example test

W-00-5 — Check for an active internship before starting

- **Change:** `src/app/api/internship/start/route.ts`, `tests/internship_eligibility.test.ts` (or a new test).
- **Steps:** Query `internship_enrollments` for this student and crash enrollment with status `generating` or `active`, and pass the real result as `activeInternship`.
- **Commit:** Refuse a second active internship

W-00-6 — Phase 0 PR

- Open a PR titled `Internship fixes (Phase 0 of the web SRS)`, stop, and report. The owner or Claude Code merges after "Web app tests" passes.

Phase 1 — Real checking infrastructure

W-01 — Add the new language values

- **Read first:** `src/lib/code/types.ts`, `src/lib/code/codeRunner.ts`, `src/components/quests/workspace/useWorkspaceState.ts` (`resolveQuestLanguage`), `src/lib/data/curriculumEnricher.ts`.
- **Steps:**
 1. Add `'typescript' | 'tsx' | 'html' | 'css'` to `CodeLanguage`.
 2. Add an optional `eLanguage` / `aLanguage` to `DayConfig`, passed through `buildEnrichedDayQuests`.
 3. `resolveQuestLanguage` returns them when set.
 4. Existing quests behave exactly as before.
- **Verify:** a test that every existing quest still resolves to its old language.
- **Commit:** Add TypeScript, TSX, HTML and CSS task languages

W-02 — JS/TS forbidden-API guard

- **Read first:** `src/lib/code/python/pythonGuard.ts`, `src/lib/server/codeWarsSandbox.ts` (`validateVmCodeSecurity`).
- **Create:** `src/lib/code/js/jsGuard.ts` (`findForbiddenJs(source)`, list from CHK-5), `tests/js_guard.test.ts`.
- **Verify:**
 - Each forbidden item is caught.
 - Normal code (`fetchUser` as a function name, `Function` inside a string in a test message) is not caught; use word boundaries.
 - Every existing JavaScript task answer and test in `tests/fixtures/practice_solutions.json` passes the guard. If some do not, list them and ask the owner before changing the list.
- **Commit:** Add forbidden-API guard for JavaScript and TypeScript tasks

W-03 — TypeScript/TSX compile step

- **Steps:**
 1. Add `esbuild-wasm` as a dependency, plus a copy script that places its `wasm` under `public/esbuild/`; hook the script into `predev` and `prebuild` as `copy-pyodide.cjs` is.
 2. Create `src/lib/code/ts/compileTs.ts`: `compileTs(source, { jsx: boolean })` returns `{ ok: true, js }` or `{ ok: false, message, line }`. It uses `esbuild-wasm` in the browser and `esbuild` in Node.
- **Verify:** `tests/ts_compile.test.ts` covers a typed function, TSX, and a syntax error that reports its line.
- **Commit:** Compile TypeScript and TSX for tasks and lessons

W-04 — React render checks

- **Steps:**
 1. Create a build script `scripts/utils/build-react-runtime.cjs` that uses `esbuild` to bundle `react` + `react-dom/server` into `public/sandbox/react-runtime.js`, exposing `globalThis.__PINIT_REACT__ = { React, renderToStaticMarkup }`.
 2. Add check helpers `render(Component, props) → HTML string`.
 3. `jsTaskScript` places the runtime in front of the student code for `tsx` tasks.
- **Verify:** `tests/react_render_check.test.ts`:

- a correct `Greeting({ name })` passes for 3 names;
- a component that ignores `name` fails;
- a constant HTML string fails.
- **Commit:** Check React components by rendering them

W-05 — HTML and CSS checks

- **Steps:** Add `parse5` and `postcss` as direct dependencies, then create `src/lib/code/web/htmlCssChecks.ts` with the helpers from CHK-4 (bundled for the Worker, like W-04).
- **Verify:** `tests/html_css_check.test.ts` checks:
 - a missing `alt` is found;
 - a label is missing;
 - headings are out of order;
 - a CSS value is read.
- **Commit:** Add HTML and CSS checks

W-06 — Wire the runners

- **Read first:** `src/lib/code/codeRunner.ts`, `src/lib/code/runners/jsTaskScript.ts`, `src/lib/code/sandbox/sandboxedIframeRunner.ts`.
- **Steps:** Route `typescript/tsx` through `compile` → `guard` → `sandbox`, and `html/css` through the check helpers. Keep the "UNTRUSTED CLIENT OBSERVATION" label.
- **Verify:** a Node test drives `runTestSuite` for each new language with a pass and a fail case.
- **Commit:** Run TypeScript, TSX, HTML and CSS tasks

W-07 — Lesson runner support

- **Change:** `src/lib/data/longLessons.ts` (`LongLessonLanguage` adds `'typescript'`), and the lesson engine example runner in `useLessonEngine.ts`.
- **Steps:** Compile TypeScript examples before running them. `adaptCodeForSandbox` must return code unchanged for quests with a set language (CHK-7).
- **Verify:** a test that an id containing `javascript` or `sql` with a set language is not rewritten.
- **Commit:** Run TypeScript lesson examples and stop rewriting web code

W-08 — Server grader for JS/TS

- **Read first:** `src/lib/server/codeWarsSandbox.ts`, `src/lib/server/pythonSandbox.ts` (after W-00-1).
- **Create:** `src/lib/server/jsSandbox.ts` (CHK-6), `tests/js_sandbox.test.ts`.
- **Verify:**
 - a correct answer passes;
 - a wrong answer fails;
 - an infinite loop times out;
 - a forbidden API is refused;
 - **printing any marker text does not pass;**

- hidden test text is never in the output.
- **Commit:** Add server grader for JavaScript and TypeScript

Phase 2 — Web track test harness

W-09 — Constant-answer detector

- **Change:** `tests/js_practice_tasks.test.ts`.
- **Steps:** Extend the "lazy answer" generator so every exported function returns **the value the check expects for the first input**. It must fail on at least one other input.
- **Report, do not fix here:** list every existing task that now fails; the fixes happen per course in Phase 3. Mark them as a known list in the test (`KNOWN_CONSTANT_TASKS`) that must shrink to empty by the end of Phase 3.
- **Commit:** Detect constant-answer practice tasks

W-10 — `tests/web_track_tasks.test.ts`

- Build it as C6 describes, driven by a `WEB_COURSES` list. It starts empty and each course adds itself when finished.
- **Commit:** Add web track task checks

W-11 — `tests/web_track_long_lessons.test.ts`

- Build it as C6 describes: run JavaScript/TypeScript examples in `vm` after `compileTs`, and run React examples with the runtime from W-04.
- **Commit:** Add web track long lesson checks

W-12 — `tests/web_track_plans.test.ts`

- Add the C6 checks, with the "12 distinct courses" and "30 long lessons each" checks listed as `todo` until Phase 4. The 24-month snapshot check is active now.
- **Commit:** Add web track plan checks

Phase 3 — Courses (in plan order)

Pattern for every course (repeat the same 10 steps; one commit per step):

Step	Task	Commit message pattern
a	Days file <code><name>WebDays.ts</code> (30 titles, desc and syllabus; C3 for new courses, existing titles for upgrades)	Add <code><Course></code> day plan
b	Tasks Days 1-15 in <code><name>Web30DayData.ts</code> , answers in <code>tests/fixtures/<name>_web_solutions.json</code> , course added to <code>WEB_COURSES</code>	Add <code><Course></code> tasks days 1-15
c	Tasks Days 16-30	Add <code><Course></code> tasks days 16-30

Step	Task	Commit message pattern
d-i	Long lessons in 6 batches of 5 days (1-5, 6-10, ..., 26-30) in <code><name>WebLongLessons.ts</code>	Add <code><Course></code> lessons days X-Y
j	Registration (C2) and removal of the course from <code>KNOWN_CONSTANT_TASKS</code>	Register <code><Course></code>

For **upgraded** courses (dsa-optim, devops, cloud, distributed, cyber, ai-eng, design):

- Keep the existing quest file and prefix, and add the long lessons.
- Rewrite only the tasks listed by W-09 (and the stubs from B3), keeping quest ids.
- Put rewritten answers in `tests/fixtures/practice_solutions.json`, replacing the old ones.

Before writing any expected value, run the reference answer and copy the real result.

Task ids	Course	Notes
W-13	<code>course-react-web</code>	Replace Day-1 <code>pageTitle</code> with a real task; convert ≥ 10 tasks to TSX render checks (W-04)
W-14 ... W-23	<code>course-node-web</code> (new, prefix <code>node-web</code>)	C3 topics; TypeScript tasks
W-24 ... W-33	<code>course-devops-cicd</code>	+ lessons
W-34 ... W-43	<code>course-cloud-native</code>	+ lessons
W-44 ... W-53	<code>course-design-systems</code>	+ lessons; rewrite 30 constant tasks with HTML/CSS/TSX checks
W-54 ... W-63	<code>course-dsa-optim</code>	+ lessons
W-64 ... W-73	<code>course-distributed-sys</code>	+ lessons; Day-30 stub
W-74 ... W-83	<code>course-cybersecurity</code>	+ lessons; rewrite 30 constant tasks
W-84 ... W-93	<code>course-ai-eng</code>	+ lessons; Day-30 stub
W-94 ... W-103	<code>course-sre-web</code> (new)	C3 topics
W-104 ... W-113	<code>course-stream-web</code> (new)	C3 topics
W-114 ... W-123	<code>course-aideploy-web</code> (new)	C3 topics

Phase 4 — Plan wiring

W-124 — Point the plans at the right courses

- **Change:** `src/lib/data/crashPlansData.ts` (web_fullstack modules only).

- **Steps:**
 1. Month 2 of 3m/6m/9m/12m → `course-node-web`.
 2. 12m Month 10 → `course-sre-web`, "Month 10: Multi-Cloud Reliability & SRE".
 3. 12m Month 11 → `course-stream-web`, "Month 11: High-Throughput Streaming".
 4. 12m Month 12 → `course-aideploy-web`, "Month 12: Production AI Deployment".
 5. Update the skills lists to match the course content.
- **Must not:** touch `plan-24m-master` or the `python_ai` modules.
- **Verify:** turn the `todo` checks in `tests/web_track_plans.test.ts` into real checks; all pass.
- **Commit:** Give every month of the web plans its own course

Phase 5 — Web capstones

W-125 — Capstone wording and promises

- **Steps:**
 - Add `capstoneSprintsByTrack.web_fullstack` for 3m-12m.
 - Review each `flagshipBuildByTrack.web_fullstack` so it lists only taught technology.
 - Extend `tests/crash_plan_promises.test.ts`.
- **Commit:** Web capstones match what each plan teaches

Phase 6 — Web internship

- **W-126** Make `src/lib/internships/tiers.ts` per-track (`INTERNSHIP_TIERS[track][tier]`), with web names from C5), and update all callers. Commit: Make internship tiers per track.
- **W-127** Remove the python-only checks: `eligibility.ts`, `InternshipPanel.tsx`, `toClient.ts`. Both tracks become eligible, subject to the per-tier switches. Commit: Allow the web track in internships.
- **W-128** Add a new migration `supabase/migrations/<date>_internship_web_track.sql`. It widens the `internship_tasks.language` check to `('python','sql','typescript','tsx')` and removes the `'python_ai'` default (use `ALTER TABLE ... DROP CONSTRAINT IF EXISTS / ADD CONSTRAINT`), and it is listed in `supabase/migrations/README.md`. Commit: Allow TypeScript internship tasks in the database.
- **W-129** Add `typescript/tsx` to the internship types, the prompt builder (`generateTask.ts`) and the validation (`validateTask.ts` `V2 = findForbiddenJs`, `V3-V5 = jsSandbox.ts`). Commit: Generate and validate TypeScript internship tasks.
- **W-130** Make `executeTicketCode` grade `typescript/tsx` with `jsSandbox.ts`, and add real-grader tests (correct passes, wrong fails, marker cheat fails, hidden source never shown). Commit: Grade TypeScript internship tickets on the server.
- **W-131** Add web Tier 1 ticket kinds (component, component bug fix, form validation, refactor, small feature) using Month 1 React skills. Commit: Add web job simulation tickets.
- **W-132** Add the web Tier 2 product brief (React + Node + PostgreSQL) and weekly tasks (1 TSX/TypeScript + 1 SQL per member per week). Commit: Add web virtual internship backlog and tasks.
- **W-133 ... W-135** Tiers 3-5 for web: **BLOCKED** (D1, D2).

- **W-136** Show the web internship screens in `InternshipPanel.tsx` with the web tier names and a TSX editor. Commit: `Show the web internship to web students`.

Phase 7 — Launch checks

- **W-137** Run the full checks, open a PR and report. **Do not** change any `INTERNSHIP_TIER_AVAILABLE` value. The owner decides each switch.
-

PART E — REFERENCE

E1. Standard commands

```
npx tsc --noEmit && npm test && npx next build
git checkout -- tsconfig.tsbuildinfo           # before every commit
pip install graphify && graphify update src      # optional code graph (never commit
src/graphify-out)
graphify query "<question>" --graph src/graphify-out/graph.json
```

E2. Key files

Purpose	File
Plans	src/lib/data/crashPlansData.ts
Course registry	src/lib/data/coursesData.ts
Day → quests	src/lib/data/curriculumEnricher.ts (buildEnrichedDayQuests)
Long lessons registry	src/lib/data/longLessons.ts
Example web course	src/lib/data/react30DayData.ts , src/lib/data/reactLongLessons.ts
Example Python pattern	src/lib/data/safetyPythonDays.ts , safetyPython30DayData.ts , safetyPythonLongLessons.ts , tests/fixtures/safety_python_solutions.json
Code runners	src/lib/code/codeRunner.ts , src/lib/code/runners/ , src/lib/code/sandbox/
Existing JS task tests	tests/js_practice_tasks.test.ts , tests/fixtures/practice_solutions.json
Lesson engine	src/app/quests/lesson/hooks/useLessonEngine.ts
Internship	src/lib/internships/ , src/app/api/internship/ , docs/internship-srs/PINIT_INTERNSHIP_SRS.md

E3. Owner's short summary (non-technical)

- **Step 1 (Phase 0):** finish the internship fixes. A cheating hole and 4 other problems are still open.
- **Then the web track:**
 - Give the 12-month web plan 12 different courses (3 new: Reliability, Streaming, AI Deployment; plus a proper 1-month Node backend course).
 - Every course gets 30 lessons of 20–30 minutes and 60 real, checked tasks.
 - The fake "return a fixed answer" tasks are replaced.
 - Students write and run real TypeScript, React, HTML and CSS.
- **Web internship:** the 1-month and 3-month levels are built; 6, 9 and 12-month levels wait for your decisions on mentors and partner companies.

- **Nothing is switched on for students without your approval.**